System Design & Architecture Masterclass

The Complete GitHub Pages Hosting, Content Protection & LinkedIn Growth Playbook

---

1. Zero-Friction Hosting on GitHub Pages

Your workspace includes a master landing portal (index.html) and 54 chapter files. Follow these exact steps to host it for free:

Step A: Push to GitHub Repository

```
git init
git add .
git commit -m 'feat: complete system design masterclass'
git branch -M main
git remote add origin https://github.com/YOUR_USER/system-design-masterclass.git
git push -u origin main
```

Step B: Enable GitHub Pages

1. Go to **GitHub Repo Settings** → **Pages**.
2. Under **Build and deployment**, select **Branch: main** and **Folder: / (root)**.
3. Click **Save**. Your site will be live at: https://YOUR_USER.github.io/system-design-masterclass/

2. Content Protection & Anti-Theft Measures

All 55 HTML files in your workspace are equipped with multi-layered client-side protection:

- **Context Menu Lock:** Disables right-click to prevent 'Save As' and element inspection.
- **DevTools Interception:** Blocks F12, Ctrl+Shift+I, Ctrl+Shift+J, Ctrl+U (view source), and Ctrl+S.
- **Selective Text Selection:** Disables general text copying while keeping code snippets selectable for genuine students.

3. LinkedIn Algorithm Mastery (How to Maximize Reach)

To get 10,000+ views and attract Staff/Principal engineers and recruiters:

1. **Never Put External Links in the Post Body:** LinkedIn penalizes posts with links by 60%. Always write: '⬛ *Link in the first comment below!* ⬛' and comment immediately after posting.
2. **Attach Visual Cards:** Posts with high-contrast infographics get 3x higher dwell time. Use the images in linkedin_post_assets/.
3. **First 60-Minute Engagement:** Reply to every comment within 30 minutes to trigger the viral recommendation loop.
4. **Optimal Posting Schedule:** Post on Tuesday, Wednesday, or Thursday at 8:00 AM or 12:30 PM local time.

4. 8 Turnkey High-Converting LinkedIn Post Templates

Post 1: The Grand Announcement (Launch Post) (Attach: linkedin_post_assets/Post_01_Grand_Launch_Hero.png)

```
■ I spent the last few weeks building an exhaustive, interactive System Design Masterclass covering 616 topics, 108 animated diagrams, and polyglot Python + TypeScript code — and I'm open-sourcing everything today! ■■

■ What's inside:
■ AI/LLM Systems: ChatGPT streaming, ColPali Multimodal RAG, Copilot FIM, Sora video latents.
■ 39 Classic & Architect Problems: TinyURL, Uber H3, Ticketmaster locks, Figma CRDTs, Stripe Ledger.
■ 11 Legendary Papers: Dynamo, Kafka, Raft, Bigtable, ZooKeeper, and Cassandra.

■ What system design topic do you find most challenging in interviews?

(■ Full interactive curriculum link in the first comment! ■)
#SystemDesign #SoftwareEngineering #DistributedSystems #ArtificialIntelligence #OpenSource
```

Post 2: Amazon Dynamo & Vector Clocks (Attach: linkedin_post_assets/Post_02_Amazon_Dynamo_VectorClocks.png)

```
■ Why does Amazon never fail when you add an item to your cart — even during data center outages?

In 2007, Amazon published the landmark Dynamo paper, optimizing for 100% write availability (AP under CAP).

4 Core Architectural Mechanisms:
1■■ Consistent Hashing with 150 Virtual Nodes keeping load variance < 5%.
2■■ Sloppy Quorums (N=3, R=2, W=2) guaranteeing write overlap.
3■■ Hinted Handoff buffering writes during transient failures.
4■■ Vector Clocks tracking causal dependencies with client cart reconciliation.

(■ Python & TypeScript Vector Clock implementation in first comment! ■)
#DistributedSystems #AmazonDynamo #SystemDesign #HighAvailability
```

Post 3: Apache Kafka Zero-Copy Throughput (Attach: linkedin_post_assets/Post_03_Apache_Kafka_ZeroCopy.png)

```
■ How does Apache Kafka stream over 1,000,000 messages/second per broker?

Kafka's secret weapon is Linux Kernel Zero-Copy and Sequential Disk I/O.

Traditional Path vs Kafka Zero-Copy:
■ Traditional: 4 context switches and 4 data copies (Disk -> OS Cache -> User JVM -> Socket -> NIC).
■ Kafka sendfile(): 2 DMA transfers directly from OS Page Cache to NIC Buffer without JVM Heap overhead!

(■ Full animated diagram & code in first comment! ■)
#ApacheKafka #Performance #Linux #BackendEngineering #SystemDesign
```

Post 4: Multimodal ColPali RAG Architecture (Attach: linkedin_post_assets/Post_04_Multimodal_ColPali_RAG.png)

```
■ Standard RAG is broken for complex PDFs (tables, charts, slides). Here is how Multimodal ColPali RAG fixes it:

1■■ No OCR: Renders 150 DPI page images directly into Vision-Language Models.
2■■ 1,024 Patch Embeddings: Captures visual quadrants preserving charts.
3■■ ColBERT Late-Interaction MaxSim: Computes token-to-patch max similarity matrix without information loss.

(■ Full Python MaxSim retrieval algorithm in first comment! ■)
#ArtificialIntelligence #LLM #RAG #MachineLearning #GenerativeAI
```